

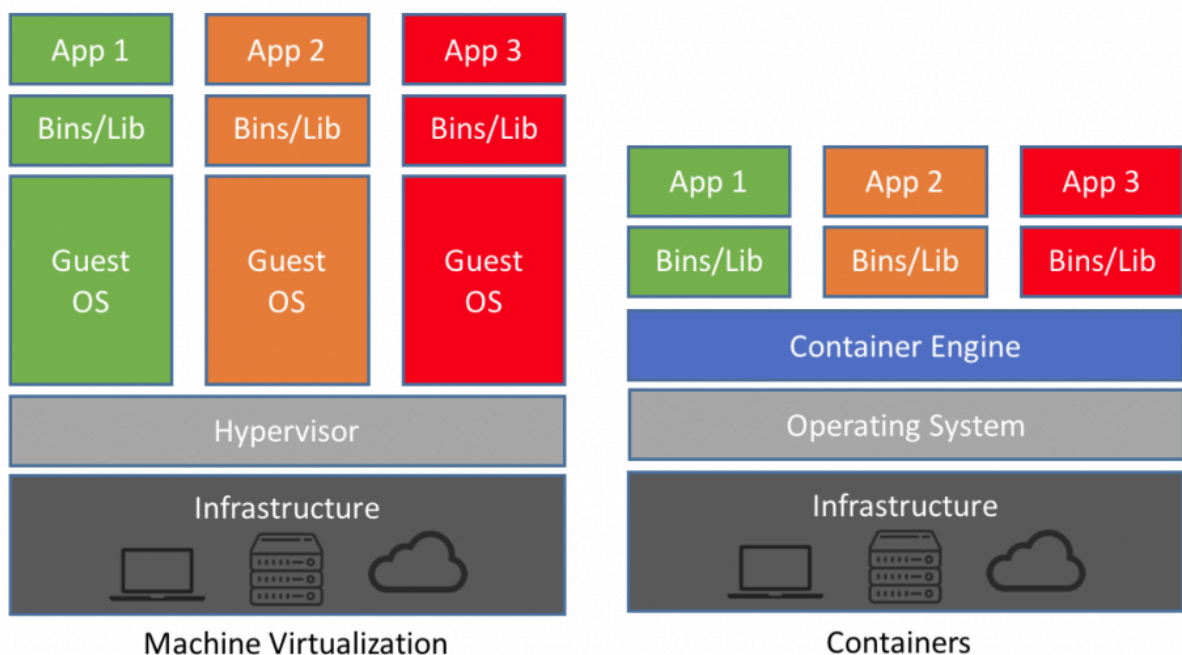
Docker Tutorial for Beginners: Basics, Architecture, Containers

By David Taylor ⌚ Updated July 15, 2023

What is Docker?

Docker is a software development platform for virtualization with multiple Operating systems running on the same host. It helps to separate infrastructure and applications in order to deliver software quickly. Unlike Hypervisors, which are used for creating VM (Virtual machines), virtualization in Docker is performed on system-level, also called Docker containers.

As you can see the difference in the image below, Docker containers run on top of the host's Operation system. This helps you to improves efficiency and security. Moreover, we can run more containers on the same infrastructure than we can run Virtual machines because containers use fewer resources.



Virtualization in Docker vs Hypervisor

Unlike the VMs which can communicate with the hardware of the host (ex: Ethernet adapter to create more virtual adapters) Docker containers run in an isolated environment on top of the host's OS. Even if your host runs Windows OS, you can have Linux images running in containers with the help of Hyper-V, which automatically creates small VM to virtualize the system's base image, in this case, [Linux](#).

In this Docker tutorial for beginners, you will learn Docker basics like:

- [What is Docker?](#)
- [Why use Docker?](#)
- [Docker Architecture](#)
- [Docker Engine](#)
- [Docker Images](#)
- [Docker Registries](#)
- [Docker Containers](#)
- [How to install Docker on Linux/Ubuntu](#)
- [How to use Docker using basic Docker Commands](#)
- [What is Virtualization?](#)
- [Important Docker Commands](#)

Why use Docker?

- Docker is computer software used for Virtualization in order to have multiple Operating systems running on the same host
- Docker is the client-server type of application which means we have clients who relay to the server
- Docker images are the “source code” for our containers; we use them to build
- Dockerfile has two types of registries 1.) public and 2) private registries
- Containers are the organizational units of Docker volume. In simple terms, an image is a template, and a container is a copy of that template. You can have multiple containers (copies) of the same image.

Docker Architecture

Now in this Docker container tutorial, let's talk about Docker main components in the Docker Architecture:

Docker Architecture

Docker Engine

Docker is the client-server type of application which means we have clients who relay to the server. So the Docker daemon called: `dockerd` is the Docker engine which represents the server. The docker daemon and the clients can be run on the same or remote host, and they communicate through command line client binary, as well as a full RESTful API to interact with the daemon: `dockerd`.

Docker Images

Docker images are the “source code” for our containers; we use them to build containers. They can have software pre-installed which speeds up deployment. They are portable, and we can use existing images or build our own.

Docker Registries

Docker stores the images we build in registries. There are public and private registries. Docker company has public registry called [Docker hub](#), where you can also store images privately. Docker hub has millions of images, which you can start using now.

Docker Containers

Containers are the organizational units and one of the Docker basics concept. When we build an image and start running it; we are running in a container. The container analogy is used because of the portability of the software we have running in our container. We can move it, in other words, “ship” the software, modify, manage, create or get rid of it, destroy it, just as cargo ships can do with real containers.

In simple terms, an image is a template, and a container is a copy of that template. You can have multiple containers (copies) of the same image.

Below we have an image which perfectly represents the interaction between the different components and how Docker container technology works.

What we have several dockers commands `docker pull`, `docker run`.. we will talk more about that later.

How to install Docker on Linux/Ubuntu

Below is a step by step Docker installation process on Linux/Ubuntu:

Step 1) To install Docker, we need to use the Docker team’s DEB packages.

For that, Some prerequisite Ubuntu packages are required.

Use the below command to install Ubuntu packages

```
$ sudo apt-get install \
apt-transport-https \
ca-certificates curl \
software-properties-common
```

***the sign “\” is not necessary it’s used for the new line, if want you can write the command without using “\” in one line only.**

Step 2) Add the official Docker GPG key with the fingerprint.

Use the below Docker command to enter the GPG key

```
$ curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo apt-key add -
```

Step 3) Next, Add the Docker APT repository.

Use the below Docker command to add the repository

```
$ sudo add-apt-repository \
"deb [arch=amd64] https://download.docker.com/linux/ubuntu \
$(lsb_release -cs) \
stable"
```

You may be prompted to confirm that you wish to add the repository and have the GPG key automatically added to your host.

The lsb_release command should populate the Ubuntu distribution version of your host.

Step 4) After adding the GPG key,

Update APT sources using the below Docker command

```
$ sudo apt-get update
```

We can now install the Docker package itself.

Step 5) Once the APT sources are updated,

Start installing the Docker packages on Ubuntu using the below Docker command

```
$ sudo apt-get install docker-ce
```

The above-given command installs Docker and other additional required packages. Before Docker 1.8.0, the package name was lxc-docker, and between Docker 1.8 and 1.13, the package name was docker-engine.

NOTE: Docker for Windows requires Windows 10 Pro or Enterprise version 14393, or Windows server 2016 RTM to run

How to use Docker using basic Docker Commands

Here is how to use Docker using basic Docker commands:

The most basic command we must run after installing Docker is `$ docker info` as we said previously.

```
$ sudo docker info
```

You should get the similar or following result

As we can see in the above Docker example, we have information about docker containers how many are running, paused or stopped and how many images we have downloaded. So let's get our first image in this Docker commands tutorial.

```
$ sudo docker pull alpine
```

With this command we are telling docker to download the image alpine, to pull it from the public registry, the latest version which is set by default.

*alpine is a minimal Docker image based on Alpine Linux with a complete package index and only 5 MB in size.

If we want to run the image as a container, we will use the following command in this Docker tutorials guide.

```
$ sudo docker run -i -t alpine /bin/bash
```

If we run the command, we will be sent directly to the alpine's terminal. The -i flag keeps STDIN open from the container, even when you are not attached to it. This persistent standard input is one half of what you require for an interactive shell. The -t flag is the other half and which instructs Docker to assign a pseudo-

tty to the container. This offers us an interactive shell in the new container. We exit the container with a simple exit command.

Now in this Docker basics tutorial, we can try running an Ubuntu image.

```
$ sudo docker run -it ubuntu /bin/bash
```

You can notice docker checks for the image locally, and if it's not there, the image is pulled from the image library automatically, and once again we have an interactive shell running. We can also name the containers as we run them.

```
$ sudo docker run --name our_container -it ubuntu /bin/bash
```

and we exit again.

We can also run container we previously created, without an interactive shell.

```
$ sudo docker start container_name
```

And stop the container writing `docker stop container_name`

```
$ sudo docker stop container_name
```

If we want to see all running containers, we just run

```
$ docker ps
```

And for all containers we add “- a” at the end of this same command, like this `docker ps -a`.

This command shows Container's ID, which image is using when was created, running status, exposed ports and randomly generated name for the container for easier management.

When we run containers, we would also like to know how much resources they are using, for that purpose we can use the command.

```
$ docker stats
```

You can also see which images we have downloaded locally and info about them.

```
$ sudo docker images
```

The command in the above Docker example displays the docker image with a tag which shows our image version, a distinctive image ID, when was created and image size.

What is Virtualization?

Earlier, the process for deploying a service was slow and painful. First, the developers were writing code; then the operations team would deploy it on bare metal machines, where they had to look out for library versions, patches, and language compilers for the code to work. If there were some bugs or errors, the process would start all over again, the developers would fix it, and then again the operational team was there to deploy.

There was an improvement with the creation of Hypervisors. Hypervisors have multiple Virtual machines or VMs on the same host, which may be running or turned off. VMs decreased the waiting time for deploying code and bug fixing in a big manner, but the real game changer was Docker containers.

Important Docker Commands

Below are the important Docker commands:

Command	Description
<code>docker info</code>	Information Command
<code>docker pull</code>	Download an image
<code>docker run -i -t image_name /bin/bash</code>	Run image as a container
<code>docker start our_container</code>	Start container
<code>docker stop container_name</code>	Stop container
<code>docker ps</code>	List of all running containers
<code>docker stats</code>	Container information
<code>docker images</code>	List of images downloaded
Docker Cleanup	Kill all running containers.

Also, read [Docker job interview questions and answers](#) for fresher as well as experienced professionals.

Summary

- Docker is a software development platform for virtualization with multiple Operating systems running on the same host. It helps to separate infrastructure and applications in order to deliver software quickly.
- **Docker Architecture:** Docker Engine, Docker Images, Docker Registries, Docker Containers.
- **Docker Engine:** Docker is the client-server type of application which means we have clients who relay to the server. So the Docker daemon called: dockerd is the Docker engine which represents the server.

- **Docker Images:** Docker images are the “source code” for our containers; we use them to build containers. They can have software pre-installed which speeds up deployment. They are portable, and we can use existing images or build our own.
- **Docker Registries:** Docker stores the images we build in registries. There are public and private registries. Docker company has public registry called Docker hub, where you can also store images privately. Docker hub has millions of images, which you can start using now.
- **Docker Containers:** Containers are the organizational units and one of the Docker basics concept. When we build an image and start running it; we are running in a container. The container analogy is used because of the portability of the software we have running in our container.

You Might Like:

- [9 BEST ITSM Tools & Software \(Open Source\) in 2023](#)
- [10+ Best IT Automation Software Tools \(Aug 2023\)](#)
- [What is DevOps? Full Form, Meaning, Principles & Examples](#)
- [DevOps Lifecycle: Different Phases Explained with Examples](#)
- [7 BEST AnyDesk Alternatives \(2023\)](#)

[Prev](#)[Report a Bug](#)[Next](#)

About

[About Us](#)

[Advertise with Us](#)

[Write For Us](#)

[Contact Us](#)

Career Suggestion

[SAP Career Suggestion Tool](#)

[Software Testing as a Career](#)

Interesting

[eBook](#)

[Blog](#)

[Quiz](#)

[SAP eBook](#)

Execute online

[Execute Java Online](#)

[Execute Javascript](#)

[Execute HTML](#)

[Execute Python](#)

© Copyright - Guru99 2023 [Privacy](#)
[Policy](#) | [Affiliate Disclaimer](#) | [ToS](#)